Lecture 07: Arrays-2

SE115: Introduction to Programming I

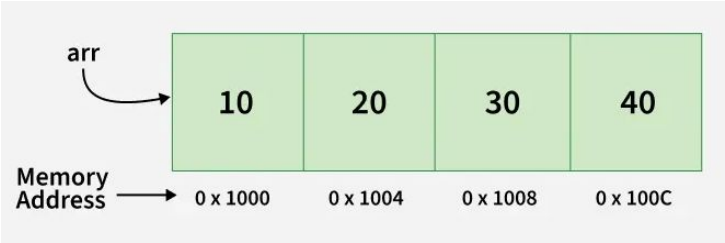
Previously on SE 115…

- Arrays

```
int[] arr = new int [4];

for(int i = 0; i < arr.length; i++){
    arr[i] = scn.nextInt();
}


for(int i = 0; i < arr.length; i++){
    System.out.println(arr[i]);
}
```



Passing Arrays as Function Parameters

- Let’s look at this week’s scenario: in an array of integer values, find if a value exists or not. The array is not in order.
- This is a straightforward “linear search” (an algorithm!).
- Think of it as a shuffled deck of cards and you are looking for a specific card, such as ace of hearts.
- The way to do it is to look at each card one by one.
- Let’s perform the search in a function, and pass the array and the searched value. This function will return a boolean value depending on whether the searched value exists in the array or not.

```java
import java.util.Scanner;
import java.util.Random;

public class ArraySearch {
    public static boolean search(int[] arr, int val) {
        for(int i=0;i<arr.length;i++) {
            if (arr[i] == val) return true;
        }
        return false;
    }

    public static void main(String[] args) {
        int size = 100;
        int range = 1000;
        Random r = new Random(System.currentTimeMillis());
        Scanner sc = new Scanner(System.in);
        int[] values = new int[size];
        for(int i=0;i<size;i++) {
            values[i] = r.nextInt(range);
        }
        System.out.print("Enter a value to search: ");
        int target = sc.nextInt();

        if(search(values, target)) {
            System.out.println(target + " exists in the array");
        } else {
            System.out.println(target + " does not exist in the array");
        }
    }
}
```

We pass the array as a parameter as we would any variable, but please notice the brackets [] - similar to the args in main.

If the function does not return true here, it will return false here.

These values are defined as variables, not magic numbers.

The search function returns a boolean value, and depending on that we vary the output to the user. This is an example of “pass by reference” - we will discuss it in the upcoming lectures.

Passing Values

- Even though the array is passed like a primitive variable, the value we pass is not the values in the array, but rather its reference.

- Which means the original array can be modified through this…

Passing Values

```java
public class ArrayArgumentExample {

    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5};

        System.out.println("Before method call:");
        printArray(numbers);

        multiplyByTwo(numbers);   // passing array to method

        System.out.println("After method call:");
        printArray(numbers);
    }

    public static void multiplyByTwo(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            arr[i] = arr[i] * 2;    // modifies original array
        }
    }

    public static void printArray(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
        System.out.println();
    }
}
```

Passing Values

- We will learn more about these “references” after the midterm, when we learn more about the objects.

Returning an array?

- Can we also return an array?
- Why, yes, of course we can.
- Here is how.

Returning an array…

```
import java.util.Random;

public class CreateArray {
    public static int[] getArray(int size, int range) {
        Random r = new Random(System.currentTimeMillis());
        int[] arr = new int[size];
        for(int i=0;i<arr.length;i++) {
            arr[i] = r.nextInt(range);
        }
        return arr;
    }

    public static void main(String[] args) {
        int[] myArray = getArray(100, 500);
        for(int i=0;i<myArray.length;i++) {
            System.out.println(myArray[i]);
        }
    }

}
```

Explain what is happening here! Volunteers?

Multidimensional Arrays

- Similar to having nested loops, we can have arrays of arrays.
- Regular arrays are 1D. Let’s try 2D.
- Initialization is the same; to access each element we can use a nested **for** loop.

Multidimensional Arrays

```
public class MultiDim {
    public static void main(String[] args) {
        int counter = 0;
        int[][] matrix = new int[5][6];
        for(int i=0;i<matrix.length;i++) {
            System.out.println("Working on matrix[" + i +"] with length " + matrix[i].length);
            for(int j=0;j<matrix[i].length;j++) {
                matrix[i][j] = counter++;
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

This means I'll have 5 arrays, each of these arrays will have 6 elements.

I'm looping in elements of the matrix, which is an array, where each element is another array.

I'm looping in elements of the matrix[i], which is an array, where each element is an integer.

Print the elements of the inner array side by side, using print, rather than println.

Move on to the next line.

Multidimensional Arrays

```java
public class MultiDim {
    public static void main(String[] args) {
        int counter = 0;
        int[][] matrix = new int[5][6];
        for(int i=0;i<matrix.length;i++) {
            System.out.println("Working on matrix[" + i +"] with length " + matrix[i].length);
            for(int j=0;j<matrix[i].length;j++) {
                matrix[i][j] = counter++;
                System.out.print(matrix[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

loops 5 times

loops 6 times

0

1

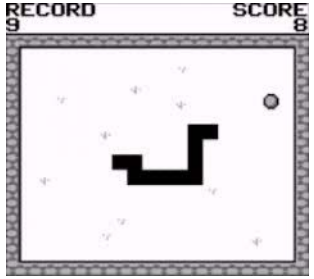
2

3

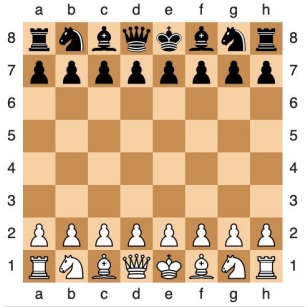
4

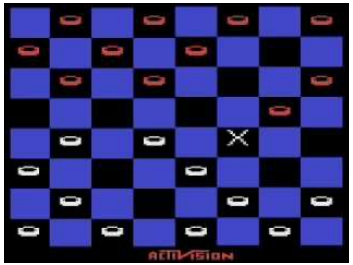
```
❯ javac MultiDim.java
❯ java MultiDim
Working on matrix[0] with length 6
0 1 2 3 4 5
Working on matrix[1] with length 6
6 7 8 9 10 11
Working on matrix[2] with length 6
12 13 14 15 16 17
Working on matrix[3] with length 6
18 19 20 21 22 23
Working on matrix[4] with length 6
24 25 26 27 28 29
```

Multidimensional Arrays

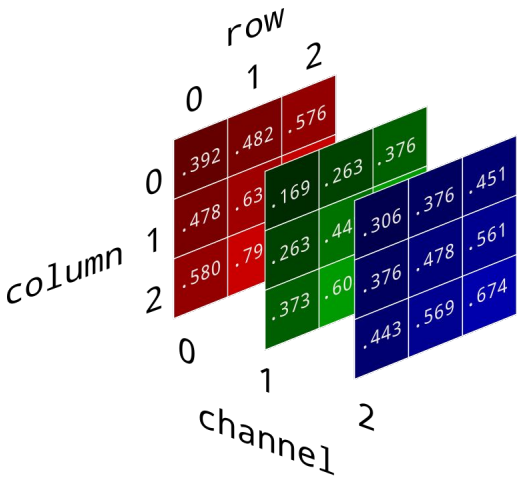


- Multidimensional arrays are very useful.
- They are commonly used in several applications, including games.
- For example, the famous Snake game is made up of a 2D matrix, and a 1D snake.
    - Snake in this case is an array of 2D points, which we will study after the midterm.
- Other traditional games, such as checkers, go, chess, backgammon, all require multidimensional arrays.





Multidimensional Arrays



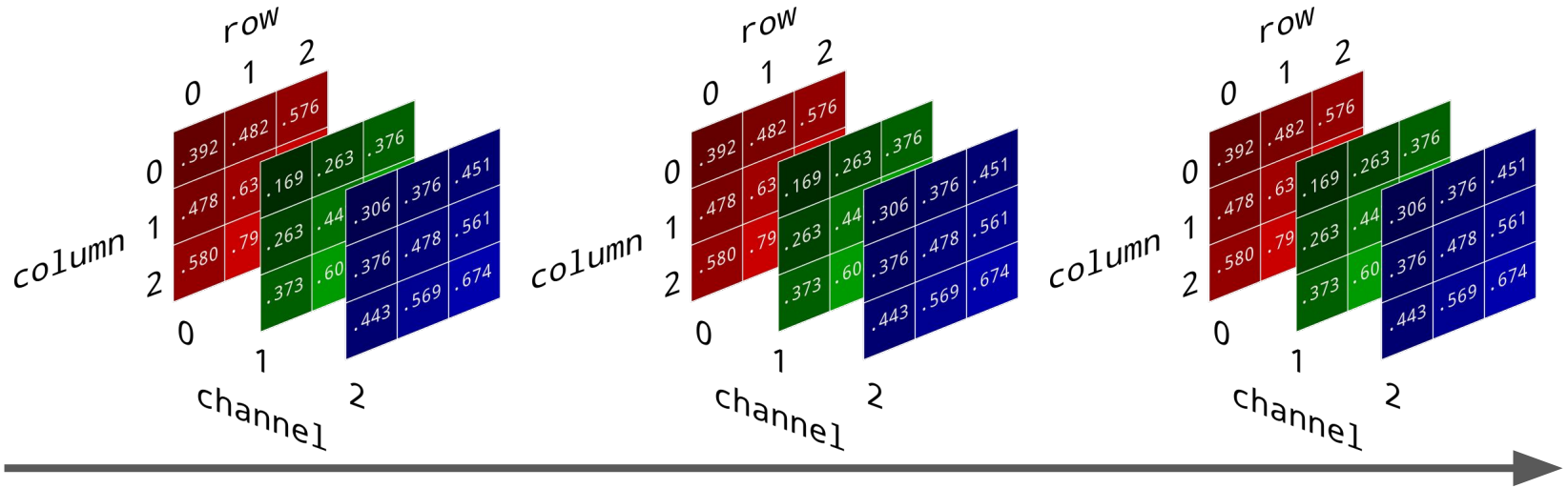
- You can have higher dimensions, too.

```
int [][][] my3d = new int[800][600][3];
```

- The variable above is a 3D array.
- When you take a photograph, it basically looks like 2D, where 800 and 600 denote the width and the height.
- To get a color image however, you need to have Red Green and Blue channels.
- The third dimension in `my3d` variable states that we have 3 of these 800x600 images which can be used to display a colored photograph.

Multidimensional Arrays

- Now add another dimension to the photograph, and it becomes a movie.
- An RGB image at a specific time!
- The fourth dimension is time: https://www.youtube.com/watch?v=PWubX3NXq5k



Midterm

- That is all about arrays!
- Next week: Midterm.
  - All the topics we have covered so far, included.
  - Multiple choice questions followed up by 2 classic ones.
- Any questions before the midterm?